

Chat Application System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

A chat application combines two distinct technical challenges: low-latency message delivery (real-time push) and durable message storage (never lose a message). PostgreSQL serves as the durable store; LISTEN/NOTIFY provides the real-time delivery mechanism for small-to-medium deployments; Redis Pub/Sub or Kafka replaces it at larger scales. This document covers the complete system design, focusing on the PostgreSQL implementation of persistence, pagination, read receipts, and presence — with clear guidance on when each piece must be offloaded.

Requirements

Functional Requirements

- 1:1 and group messaging
- Real-time message delivery via WebSocket
- Persistent message history
- Read receipts (delivered and seen)
- Typing indicators (ephemeral, no storage)
- Presence (online/offline)
- File/media attachments
- Message reactions
- Push notifications for offline users
- Conversation search

Non-Functional Requirements

- 200M users, 50M DAU
- 10B messages per day
- Sub-100ms delivery for online users
- Messages stored indefinitely
- Exactly-once delivery guarantee (deduplication)
- At-least-once notification for offline users

- 99.99% uptime for message send

Capacity Estimation

Metric	Value
Daily active users	50M
Messages per day	10B
Peak messages per second	200K
Average message size	200B (text only)
Average conversations per user	30
Total active conversations	750M
Read receipts per day	50B (5 per msg avg)
Presence heartbeats per second	1.7M (50M DAU / 30s)

Storage Estimates

Component	Technology	Data Size
Messages (hot, last 90 days)	PostgreSQL	$10\text{B/day} \times 200\text{B} \times 90 = 180\text{TB}$
Messages (archive)	Cassandra / S3	$10\text{B/day} \times 200\text{B} = 2\text{TB/day}$
Read receipts (hot)	PostgreSQL (30 days)	$50\text{B} \times 30\text{d} \times 80\text{B} = 120\text{TB}$
Conversation metadata	PostgreSQL	~50GB
Presence	Redis	~5GB (50M users)
Attachment metadata	PostgreSQL	~10GB/day

Critical decision: 10B messages/day at 200B/message = 2TB/day. PostgreSQL handles up to ~10TB comfortably per node. Beyond 90 days (180TB), archive to Cassandra or object storage.

Schema Design

```
-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- =====
-- USERS AND PRESENCE
-- =====
CREATE TABLE users (
```

```

    user_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    phone            VARCHAR(30)    NOT NULL,
    display_name     VARCHAR(200)   NOT NULL,
    avatar_url       VARCHAR(500),
    about            TEXT,
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT uq_phone UNIQUE (phone)
);

-- UNLOGGED: presence is ephemeral, rebuilt from client heartbeats
CREATE UNLOGGED TABLE user_presence (
    user_id          UUID          PRIMARY KEY REFERENCES users(user_id),
    is_online        BOOLEAN       NOT NULL DEFAULT FALSE,
    last_seen_at     TIMESTAMPTZ    NOT NULL DEFAULT now(),
    device_token     VARCHAR(500), -- for push notifications
    ws_server_id     VARCHAR(50),  -- which WebSocket server they're connected to
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT now()
);

-- =====
-- CONVERSATIONS
-- =====

CREATE TYPE conv_type AS ENUM ('direct', 'group');

CREATE TABLE conversations (
    conv_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    conv_type        conv_type     NOT NULL DEFAULT 'direct',
    name             VARCHAR(500),
    description       TEXT,
    avatar_url       VARCHAR(500),
    created_by       UUID          NOT NULL REFERENCES users(user_id),
    -- Denormalized for fast inbox load
    last_message_id  UUID,
    last_message_preview VARCHAR(200),
    last_activity_at TIMESTAMPTZ    NOT NULL DEFAULT now(),
    member_count     INT           NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT now()
);

-- Ensures only one DM between a pair
CREATE TABLE direct_conversation_index (
    user_a          UUID          NOT NULL,
    user_b          UUID          NOT NULL,
    conv_id         UUID          NOT NULL REFERENCES conversations(conv_id),
    PRIMARY KEY (user_a, user_b),
    CONSTRAINT chk_canonical_order CHECK (user_a < user_b)
);

CREATE TABLE conversation_members (
    conv_id          UUID          NOT NULL REFERENCES conversations(conv_id),
    user_id          UUID          NOT NULL REFERENCES users(user_id),
    role             VARCHAR(20)   NOT NULL DEFAULT 'member',

```

```

joined_at      TIMESTAMPTZ      NOT NULL DEFAULT now(),
left_at        TIMESTAMPTZ,
is_muted       BOOLEAN          NOT NULL DEFAULT FALSE,
is_pinned      BOOLEAN          NOT NULL DEFAULT FALSE,
-- Read state tracking
last_read_at   TIMESTAMPTZ,
last_read_seq  BIGINT,          -- message sequence number
unread_count   INT              NOT NULL DEFAULT 0,
PRIMARY KEY (conv_id, user_id)
);

-- =====
-- MESSAGES (Core – Partitioned by Day for Hot Data)
-- =====

CREATE TABLE messages (
-- Composite PK: message_id + sent_at for partition routing
message_id     UUID              NOT NULL DEFAULT uuid_generate_v4(),
conv_id        UUID              NOT NULL REFERENCES conversations(conv_id),
sender_id      UUID              NOT NULL REFERENCES users(user_id),
-- Content
msg_type       VARCHAR(20)       NOT NULL DEFAULT 'text',
body           TEXT,
-- Threading
parent_id      UUID,
-- State
is_edited      BOOLEAN          NOT NULL DEFAULT FALSE,
is_deleted     BOOLEAN          NOT NULL DEFAULT FALSE,
-- Deduplication (client-generated ID)
client_msg_id  VARCHAR(100),
-- Timestamps
sent_at        TIMESTAMPTZ      NOT NULL DEFAULT now(),
edited_at      TIMESTAMPTZ,
PRIMARY KEY (message_id, sent_at),
CONSTRAINT uq_client_msg UNIQUE (client_msg_id)
) PARTITION BY RANGE (sent_at);

-- Daily partitions for hot data
CREATE TABLE messages_2024_01_01 PARTITION OF messages
FOR VALUES FROM ('2024-01-01') TO ('2024-01-02');
-- Automate: pg_partman creates/drops daily partitions
-- Archive: detach partitions older than 90 days, export to Cassandra/S3

-- =====
-- MESSAGE SEQUENCE (per conversation – for ordering)
-- =====
-- Each conversation has a monotonically increasing sequence number
-- This enables "messages after seq N" queries without timestamp ambiguity
CREATE TABLE conversation_sequences (
conv_id        UUID              NOT NULL REFERENCES conversations(conv_id),
message_id     UUID              NOT NULL,
sent_at        TIMESTAMPTZ      NOT NULL,
seq_num        BIGINT           NOT NULL,

```

```

        PRIMARY KEY (conv_id, seq_num)
    ) PARTITION BY RANGE (sent_at);

-- Sequence generator per conversation (in Redis: INCR conv:{id}:seq)
-- Written to PostgreSQL asynchronously for durability

-- =====
-- READ RECEIPTS (High Volume)
-- =====
CREATE TABLE read_receipts (
    message_id      UUID          NOT NULL,
    sent_at         TIMESTAMPTZ   NOT NULL,
    user_id         UUID          NOT NULL REFERENCES users(user_id),
    conv_id         UUID          NOT NULL,
    delivered_at    TIMESTAMPTZ,
    read_at         TIMESTAMPTZ,
    PRIMARY KEY (message_id, user_id, sent_at)
) PARTITION BY RANGE (sent_at);

CREATE TABLE read_receipts_2024_01 PARTITION OF read_receipts
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- =====
-- ATTACHMENTS
-- =====
CREATE TABLE attachments (
    attachment_id   UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    message_id      UUID          NOT NULL,
    sent_at         TIMESTAMPTZ   NOT NULL,
    conv_id         UUID          NOT NULL,
    uploader_id     UUID          NOT NULL REFERENCES users(user_id),
    file_name       VARCHAR(500)  NOT NULL,
    file_size_bytes BIGINT        NOT NULL,
    mime_type       VARCHAR(100)  NOT NULL,
    cdn_url         VARCHAR(1000) NOT NULL,
    thumbnail_url   VARCHAR(1000),
    width_px        INT,
    height_px       INT,
    duration_secs   INT,
    is_expired      BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- =====
-- REACTIONS
-- =====
CREATE TABLE reactions (
    message_id      UUID          NOT NULL,
    sent_at         TIMESTAMPTZ   NOT NULL,
    user_id         UUID          NOT NULL REFERENCES users(user_id),
    emoji           VARCHAR(50)   NOT NULL,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),

```

```

        PRIMARY KEY (message_id, user_id, emoji)
    );

-- =====
-- PUSH NOTIFICATION QUEUE
-- =====

CREATE TABLE push_notifications (
    notification_id UUID          NOT NULL DEFAULT uuid_generate_v4(),
    user_id         UUID          NOT NULL REFERENCES users(user_id),
    device_token    VARCHAR(500)  NOT NULL,
    title           VARCHAR(200),
    body            VARCHAR(500),
    data            JSONB,
    status          VARCHAR(20)   NOT NULL DEFAULT 'queued',
    attempts        SMALLINT      NOT NULL DEFAULT 0,
    scheduled_at    TIMESTAMPTZ   NOT NULL DEFAULT now(),
    sent_at         TIMESTAMPTZ,
    failed_at       TIMESTAMPTZ,
    PRIMARY KEY (notification_id, scheduled_at)
) PARTITION BY RANGE (scheduled_at);

-- =====
-- REAL-TIME: LISTEN/NOTIFY TRIGGER
-- =====

CREATE OR REPLACE FUNCTION notify_message_sent()
RETURNS TRIGGER AS $$
DECLARE
    payload TEXT;
BEGIN
    payload := json_build_object(
        'event', 'new_message',
        'message_id', NEW.message_id,
        'conv_id', NEW.conv_id,
        'sender_id', NEW.sender_id,
        'msg_type', NEW.msg_type,
        'body', CASE WHEN NEW.is_deleted THEN NULL ELSE LEFT(NEW.body, 200) END,
        'sent_at', NEW.sent_at
    )::TEXT;

    -- Notify channel per conversation (WebSocket servers listen to this)
    PERFORM pg_notify('conv_' || replace(NEW.conv_id::TEXT, '-', ''), payload);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_notify_message
AFTER INSERT ON messages
FOR EACH ROW EXECUTE FUNCTION notify_message_sent();

-- =====
-- TYPING INDICATORS (no DB, pure Redis Pub/Sub)

```

```
-- =====
-- Application sends: PUBLISH typing:{conv_id} {user_id}:{action}
-- WebSocket server subscribes: SUBSCRIBE typing:{conv_id}
-- No database writes for typing – entirely ephemeral

-- =====
-- PAGINATION HELPER: Cursor-Based Pagination
-- =====
-- Always use sent_at as cursor (not OFFSET) for message history
-- Client stores: last_loaded_sent_at = oldest message in view
-- Next page query: WHERE sent_at < $cursor ORDER BY sent_at DESC LIMIT 50
```

ASCII ER Diagram

```
+-----+      +-----+      +-----+
|  users  |1-----N|conv_membrs|N-----1|   conv   |
+-----+      +-----+      +-----+
      |1              |1
      |N              |N
+-----+      +-----+      +-----+
|user_prsnc|      | messages |1-----N|read_rcpts|
+-----+      +-----+      +-----+
                        |1
                        |N
                        +-----+
                        | attachments|
                        +-----+
                        | reactions |
                        +-----+

+-----+      +-----+
|  users  |      | direct_conversation_index |
+-----+      | (user_a, user_b → conv_id)|
                  +-----+

+-----+      +-----+
|  conv   |      | conv_seqs |
+-----+      +-----+
(per-conv monotonic sequence)
```

Indexing Strategy

```
-- Users: phone lookup (registration and contact sync)
CREATE INDEX idx_users_phone      ON users (phone);

-- Conversation members: user's conversation list (inbox)
CREATE INDEX idx_members_user      ON conversation_members
  (user_id, last_activity_at DESC)
  WHERE left_at IS NULL;
```

```

-- Conversation members: members of a conversation
CREATE INDEX idx_members_conv      ON conversation_members
    (conv_id, joined_at)
    WHERE left_at IS NULL;

-- Messages: conversation timeline (most frequent read)
CREATE INDEX idx_messages_conv_time ON messages (conv_id, sent_at DESC)
    WHERE NOT is_deleted;

-- Messages: sender's sent messages
CREATE INDEX idx_messages_sender   ON messages (sender_id, sent_at DESC);

-- Messages: thread replies
CREATE INDEX idx_messages_parent   ON messages (parent_id, sent_at ASC)
    WHERE parent_id IS NOT NULL;

-- Message sequence: sync query ("messages after seq N")
CREATE INDEX idx_conv_seq          ON conversation_sequences (conv_id, seq_num
ASC);

-- Read receipts: unread query per user
CREATE INDEX idx_receipts_user     ON read_receipts (user_id, conv_id, sent_at
DESC);
CREATE INDEX idx_receipts_message  ON read_receipts (message_id);

-- Reactions: per message emoji summary
CREATE INDEX idx_reactions_msg     ON reactions (message_id, emoji);

-- Attachments: media library per conversation
CREATE INDEX idx_attachments_conv  ON attachments (conv_id, created_at DESC);

-- Push notifications: worker queue
CREATE INDEX idx_push_queued       ON push_notifications
    (status, scheduled_at, attempts)
    WHERE status = 'queued' AND attempts < 3;

-- Presence: recently active users (for "online" indicators)
CREATE INDEX idx_presence_updated  ON user_presence (updated_at DESC)
    WHERE is_online = TRUE;

```

Query Patterns

Q1 — Load Conversation Inbox

```

SELECT
    c.conv_id, c.conv_type, c.name, c.avatar_url,
    c.last_activity_at, c.last_message_preview,
    cm.unread_count, cm.is_muted, cm.is_pinned,
    up.is_online AS partner_online  -- for DMs: other person's presence
FROM conversation_members cm

```



```

JOIN conversations c ON c.conv_id = cm.conv_id
LEFT JOIN direct_conversation_index dci ON dci.conv_id = c.conv_id
LEFT JOIN user_presence up ON up.user_id = CASE
    WHEN dci.user_a = $current_user_id THEN dci.user_b
    ELSE dci.user_a
END
WHERE cm.user_id = $current_user_id
    AND cm.left_at IS NULL
ORDER BY cm.is_pinned DESC, c.last_activity_at DESC
LIMIT 50;

```

Q2 — Load Message History (Cursor-Based)

```

SELECT
    m.message_id, m.sender_id, m.msg_type, m.body,
    m.is_edited, m.is_deleted, m.sent_at,
    m.parent_id,
    u.display_name, u.avatar_url,
    json_agg(DISTINCT jsonb_build_object('emoji', r.emoji,
        'count', (SELECT COUNT(*) FROM reactions r2
            WHERE r2.message_id = m.message_id AND r2.emoji = r.emoji)
        )) FILTER (WHERE r.emoji IS NOT NULL) AS reactions,
    json_agg(DISTINCT jsonb_build_object(
        'url', a.cdn_url, 'type', a.mime_type, 'name', a.file_name
    )) FILTER (WHERE a.attachment_id IS NOT NULL) AS attachments
FROM messages m
JOIN users u ON u.user_id = m.sender_id
LEFT JOIN reactions r ON r.message_id = m.message_id
LEFT JOIN attachments a ON a.message_id = m.message_id
WHERE m.conv_id = $conv_id
    AND m.sent_at < $cursor           -- cursor for backward pagination
    AND m.sent_at >= $cursor_floor    -- partition pruning hint
    AND m.parent_id IS NULL          -- top-level messages
GROUP BY m.message_id, u.user_id
ORDER BY m.sent_at DESC
LIMIT 50;

```

Q3 — Send Message (with LISTEN/NOTIFY trigger)

```

BEGIN;

INSERT INTO messages (
    message_id, conv_id, sender_id, msg_type, body, client_msg_id, sent_at
) VALUES (
    uuid_generate_v4(), $conv_id, $sender_id, $type, $body, $client_msg_id, now()
)
ON CONFLICT (client_msg_id) DO NOTHING -- idempotent retry
RETURNING message_id, sent_at;
-- TRIGGER fires pg_notify automatically

```

```

-- Update conversation
UPDATE conversations
SET last_message_id      = $message_id,
    last_message_preview = LEFT($body, 200),
    last_activity_at     = now()
WHERE conv_id = $conv_id;

-- Increment unread for other members
UPDATE conversation_members
SET unread_count = unread_count + 1
WHERE conv_id = $conv_id
    AND user_id != $sender_id
    AND left_at IS NULL
    AND NOT is_muted;

COMMIT;

```

Q4 — Mark Conversation as Read

```

BEGIN;

-- Bulk read receipts
INSERT INTO read_receipts (message_id, sent_at, user_id, conv_id, read_at)
SELECT m.message_id, m.sent_at, $user_id, $conv_id, now()
FROM messages m
WHERE m.conv_id = $conv_id
    AND m.sent_at > $since_time
    AND m.sent_at >= now() - INTERVAL '90 days' -- partition pruning
    AND m.sender_id != $user_id
ON CONFLICT DO NOTHING;

-- Reset unread counter
UPDATE conversation_members
SET unread_count      = 0,
    last_read_at      = now()
WHERE conv_id = $conv_id AND user_id = $user_id;

COMMIT;

```

Q5 — Sync Messages Since Last Online (reconnect)

```

-- Get all messages since the user's last sequence number
SELECT
    m.message_id, m.conv_id, m.sender_id,
    m.msg_type, m.body, m.sent_at, m.is_deleted,
    cs.seq_num
FROM conversation_sequences cs
JOIN messages m

```

```

    ON m.message_id = cs.message_id
    AND m.sent_at = cs.sent_at
WHERE cs.conv_id = $conv_id
    AND cs.seq_num > $last_known_seq
    AND cs.sent_at >= now() - INTERVAL '90 days' -- partition hint
ORDER BY cs.seq_num ASC
LIMIT 200;

```

Q6 — Full-Text Search Within a Conversation

```

SELECT
    m.message_id, m.body, m.sent_at,
    u.display_name, u.avatar_url,
    ts_headline('english', m.body,
        to_tsquery('english', $query), 'MaxWords=15') AS snippet
FROM messages m
JOIN users u ON u.user_id = m.sender_id
WHERE m.conv_id = $conv_id
    AND to_tsvector('english', m.body) @@ to_tsquery('english', $query)
    AND NOT m.is_deleted
ORDER BY m.sent_at DESC
LIMIT 20;

```

Q7 — Presence Heartbeat Update

```

INSERT INTO user_presence (user_id, is_online, last_seen_at, device_token,
ws_server_id, updated_at)
VALUES ($user_id, TRUE, now(), $device_token, $ws_server_id, now())
ON CONFLICT (user_id) DO UPDATE
    SET is_online      = TRUE,
        last_seen_at   = now(),
        device_token    = EXCLUDED.device_token,
        ws_server_id    = EXCLUDED.ws_server_id,
        updated_at      = now();

```

Q8 — Offline Push Notification Queue

```

-- Get online status for conversation members
SELECT
    cm.user_id,
    up.is_online,
    up.device_token,
    up.last_seen_at
FROM conversation_members cm
LEFT JOIN user_presence up ON up.user_id = cm.user_id
WHERE cm.conv_id = $conv_id
    AND cm.user_id != $sender_id
    AND cm.left_at IS NULL

```

```

    AND NOT cm.is_muted;

-- For offline users: enqueue push notification
INSERT INTO push_notifications (user_id, device_token, title, body, data)
VALUES ($user_id, $token, $title, $preview, $payload_json);

```

Q9 — Push Notification Worker (SKIP LOCKED)

```

UPDATE push_notifications
SET status = 'sending',
    attempts = attempts + 1
WHERE notification_id IN (
    SELECT notification_id FROM push_notifications
    WHERE status = 'queued'
        AND scheduled_at <= now()
        AND attempts < 3
    ORDER BY scheduled_at ASC
    FOR UPDATE SKIP LOCKED
    LIMIT 100
)
RETURNING notification_id, user_id, device_token, title, body, data;

```

Q10 — Stale Presence Cleanup (runs every 60 seconds)

```

UPDATE user_presence
SET is_online = FALSE
WHERE is_online = TRUE
    AND updated_at < now() - INTERVAL '45 seconds';
-- Returns count of users who went offline

```

Scaling Strategy

Phase 1: Single Node (0-5M users)

- PostgreSQL (LISTEN/NOTIFY), Redis for presence, WebSocket servers
- LISTEN/NOTIFY delivers messages to all WebSocket servers (they filter by conversation)
- Messages partitioned daily; archive after 90 days

Phase 2: Horizontally Scaled WebSocket Layer (5M-50M users)

```

Client → Load Balancer → WebSocket Server (stateless, N instances)
                        |
                        LISTEN conv_{id} on PostgreSQL Primary
                        OR
                        Redis PubSub SUBSCRIBE conv_{id}
                        |
                        Only the WS server that has the client
                        for this conversation receives and forwards

```

PostgreSQL LISTEN/NOTIFY limitation: All WebSocket servers must LISTEN to the same PostgreSQL primary. At 50M DAU × 30 open conversations = 1.5B LISTEN channels — not feasible. Move to Redis Pub/Sub at this scale.

Phase 3: Redis Pub/Sub (50M–200M users)

- WebSocket server subscribes to Redis channels for conversations the user has open
- On message INSERT to PostgreSQL, application (or trigger) publishes to Redis
- Redis Pub/Sub scales horizontally (cluster mode)
- PostgreSQL receives ~200K writes/second (message inserts); handles with partitioning + batching

Phase 4: Cassandra for Message Storage (200M+ users)

- Messages table migrated to Cassandra (partition key = conv_id, clustering key = sent_at DESC)
- PostgreSQL retains: conversation metadata, members, read receipts, presence
- Cassandra handles: message storage, message history reads
- Message delivery: Redis Pub/Sub (no database reads in hot path)

Tradeoffs

Decision	Chosen	Alternative	Why
LISTEN/NOTIFY for real-time	pg_notify trigger	Polling	Eliminates polling — push model reduces latency from seconds to milliseconds
UNLOGGED presence	UNLOGGED table	WAL-logged	5x faster writes; presence rebuilt from heartbeats after crash
Daily message partitions	Daily	Monthly	10B messages/day × 200B = 2TB/day — daily partitions enable fast archival
Cursor pagination	sent_at cursor	OFFSET	OFFSET skips rows incorrectly when new messages arrive; cursor is stable
Denormalized unread_count	Counter column	COUNT query	COUNT(*) on billions of read receipts per inbox load is unacceptable
client_msg_id dedup	UNIQUE constraint	Application-level	Database-level idempotency is more reliable than application-level checks

Interview Discussion Points

1. **How do you guarantee message ordering?** Within a conversation, PostgreSQL assigns seq_num via Redis INCR (atomic, monotonically increasing per conversation). Client reconnects sync using WHERE seq_num > \$last_known_seq . The sequence prevents gaps and ordering ambiguity from concurrent inserts with the same millisecond timestamp.
2. **How does LISTEN/NOTIFY work here?** The INSERT trigger on messages calls pg_notify('conv_{id}', payload) . All WebSocket server processes that have issued LISTEN conv_{id} receive the payload on their PostgreSQL connection. Each WS server delivers it to the connected client(s) for that conversation. No polling; sub-millisecond delivery.

3. **How do you handle message deduplication?** The `client_msg_id` column has a UNIQUE constraint. The client generates a UUID for each message before sending. If the network fails after the server inserts but before the client receives the ACK, the client retries with the same `client_msg_id`. The `ON CONFLICT DO NOTHING` clause makes the retry idempotent.
4. **How do you scale to 1B users?** Move messages to Cassandra (excellent for append-heavy time-series). Keep PostgreSQL for conversation metadata, members, and settings. Move real-time delivery to Kafka (partitioned by `conv_id`, processed by WS fanout service). Redis cluster for presence and typing indicators.
-

Common Interview Follow-ups

Q: How do you implement end-to-end encryption? A: The server stores only ciphertext. Message body is encrypted client-side using Signal Protocol (double ratchet). PostgreSQL schema is identical — `body TEXT` holds base64-encoded ciphertext. Key exchange happens out-of-band (one-time prekeys published to the server). Full-text search is impossible for E2E encrypted messages.

Q: How do you handle a user deleting their account? A: Soft delete the user record. Set `display_name = 'Deleted Account'`, clear PII fields. Messages remain (with `sender_id` pointing to the deleted user record) to preserve conversation integrity. Conversations where the deleted user was the only member are archived.

Q: How do you implement group message delivery at scale? A: For groups of < 1000 members: direct fan-out (notify each member's WebSocket connection). For groups of > 1000 members: Kafka topic per group (`group.{group_id}`), partitioned by `message_id`. WebSocket servers consume from the relevant Kafka topics. PostgreSQL stores the message once; delivery is handled by the Kafka consumer layer.

Performance Considerations

- **LISTEN/NOTIFY payload limit:** 8KB max. Store only IDs + preview in the payload. Recipients fetch full message via API if needed (cache in Redis for 30 seconds to avoid thundering herd).
 - **unread_count HOT updates:** Every message to a conversation updates N rows in `conversation_members`. For large groups, batch this update using `user_id IN (SELECT user_id FROM members WHERE conv_id = $id)` in a single UPDATE.
 - **Daily partition lifecycle:** `pg_partman` creates tomorrow's partition today. `DETACH PARTITION` for partitions older than 90 days (instant, non-blocking). Export detached partitions to Cassandra or S3 asynchronously.
 - **Presence heartbeat flood:** 50M DAU × 1 heartbeat/30s = 1.7M writes/second. PostgreSQL UNLOGGED can handle ~500K UPSERTs/sec per node. Above this, Redis is mandatory for presence.
 - **Read receipt batch:** Don't insert one read receipt per message per view. Batch: when user views a conversation, insert all unread messages' receipts in a single bulk INSERT.
-

Cross-References

- See `18_Architecture_Case_Studies/06_messaging_system.md` for the detailed schema
- See `05_social_network_design.md` for notifications architecture
- See `08_system_design_framework.md` for interview presentation framework
- PostgreSQL LISTEN/NOTIFY: Chapter 10 of this guide
- UNLOGGED tables: Chapter 5 of this guide
- Partitioning: Chapter 5 of this guide